



The Lazy Developer

01

MODULE

How to build amazing software with AI

This is where your journey starts! This free module goes through a lot of foundational knowledge to help you understand what's required to build amazing software with AI if you cannot code today.

Table of Contents

- 01. What can you expect to learn here? (6 min)
- 02. The magic of LLMs and code generation. (5 min)
- 03. Treating your AI like a software developer working for you. (10 min)
- 04. What is “good” software? (11 min)
- 05. Get better results through better prompts (10 min)
- 06. Challenging Your AI and Improving Solutions (5 min)
- 07. Start with something small before you build your first Software as a Service (SaaS) product. (13 min)
- 08. Just ship! Don't waste time perfecting in the beginning (10 min)
- 09. Patience is More Than a Virtue, It's a Requirement (6 min)
- 10. Module 1 Recap: Key Takeaways and Best Practices (3 min)

Listen to this section:

Hello and welcome to the platform! This is a living, breathing set of documents and your subscription goes to helping keep this updated with new content, examples, and giving you a chance to build something!

Note: I will refer to AI as "AI", "LLM", "Cursor", and other things, these are all interchangeable as we use different tools today so don't worry about that so much. Just remember it's all AI in the end!

So who am I and why should you listen to me? Well, I'm a life-long techie, with a career in enterprise software and hardware as a sales engineer (someone in sales who is very technical). I previously had never coded beyond HTML and CSS. However, I was always involved in software and the development of that software without being in the programming team due to my career. Selling enterprise SaaS (Software as a Service) applications teaches you a lot about the requirements, the expectations from customers, the support, the profitability, and the marketing. Everything you need to do it on your own.

I always wanted to build my own apps and so I started to learn coding thanks to Cursor and the AI models helping me. I have built a few apps with Cursor and Claude Code, but my most complex app with close to 100,000 lines of code is now deployed and distributed between Vercel, Railway, and Supabase. It leverages multiple APIs as well, scrapes websites, integrates with DeepSeek AI, and provides extensive customisations. Lastly, and mostly importantly, it generates revenue! It goes to show you the power of these tools today is inspiring and great things can be achieved! That app has taken me about 3 months to build working a few hours a day but as it was my second app, it took me a bit longer to set up as I was learning cursor plus a host of new themes and principles at the same time.

If you're not technical, you will find a lot of the concepts new to you, but the goal here is to get you to START building and learn as you go along. You've got some smart AI assistants to help you on the way, but you need guidance. This is where this platform comes into play. You just have to remember one very important rule.

Just remember to be curious and ask lots of questions to the LLMs (the AI assistants).

You will find the Resources list to the left of the modules on the dashboard page very useful (or above if on mobile). There's terminology, common commands, and information to help setup your local development environment.

You will save time - hours of time - not wasting it on building poorly, debugging errors, backtracking over mistakes, and not getting lost in the weeds. When I started out I was making these mistakes all the time, but then I got smart quickly and figured out there are things I can do that will improve my ability to build proper software in less time and enjoy the process along the way. This is your platform to build software in Cursor with AI the right way!

Real-world example: Imagine you have a fantastic business idea for an app but no coding background. Using this platform and AI assistance, you can quickly prototype and develop a functional app within a few hours/days. Then, gather user feedback, iterate rapidly, and even launch commercially within weeks or a few months depending on its complexity.

Questions to consider:

- What's one app or software idea you've always wanted to build?
- What's been holding you back from starting your software-building journey?
- Are you prepared to be patient and give this a go??

Listen to this section:

(see above - a nice and tidy file structure created under some initial guidance)

You can build some significantly complicated apps with AI, but the more you develop your app, the harder it gets to manage it all. You see, AI will help you build all kinds of things rather quickly. Many AI influencers on social media say you can do amazing things in minutes! Don't believe the hype! Sure, you can mockup something really quick, but you haven't deployed it, it mostly won't work, it won't be secure, can't scale, and you haven't yet paid any money towards hosting it, or the APIs you might need to make it work. So, to do things the right way, you need to follow some structure, some process and some discipline!

It's far too easy for AI to just add lines and lines of code allowing you to lose track of what you're doing, and then you just end up with messy software that doesn't work well due to too many bugs.

Think of it like building a house and instead of fixing your errors, you just keep building more and more. The first bedroom leaks so we built another one. The next room was too small so we built another one! The windows weren't large enough in that one so we built another one.

Before you know it, you have a bloated house (application) that doesn't run well (slow), is prone to issues (bugs), and can easily fall apart (crash). It doesn't meet building codes (insecure), has lots of special quirks you have to keep track of (hard-coded addresses and settings), and it can't hold a huge house-warming party (not scalable)!

(see above) A classic example of where the LLM doesn't check before it runs a task.

So, the magic is in how you utilise these LLMs to work for you and build the way you want. If you're not familiar with building apps, don't stress, we will help you with that. We will discuss these components in future sections but to best guide you as you build. You can simply ask the LLM to help you get started! Just keep in mind that your results depend significantly on how effectively you communicate with the AI. This skill, known as "prompt engineering," is about crafting clear, precise prompts to achieve the best results. Like a conversation with a developer, the clearer your instructions, the better the output you'll get. See our following sections on this that go into that into a lot of depth.

Here's an example: Sarah, an event planner with no coding experience, wanted a straightforward app to manage event attendees. That means, a way to scan in people by their QR code when they register, categorise them by industry, and allow for a way to send out notifications and alerts to their mobile phone numbers if they opt in to let them know about training sessions and activities.

By clearly articulating her needs through a product requirements document, Sarah quickly got a working prototype from an LLM. This is because she and the LLM went through various components of the application taking into consideration the user experience, the problem they were trying to

solve, and the security needed for the app to get off the ground. Along the way, Sarah kept saying she didn't know all the questions to ask and so if the LLM would ask her any questions she could answer them with her expertise. Over several weeks, she refined her app based on feedback, and soon it became an essential tool —all without writing a single line of traditional code herself. Granted it doesn't do everything, but it was never designed too. The goal was to get a major issue resolved for her with low cost and just her desire to try and learn.

Questions to consider:

- Have you had experience crafting instructions or prompts for an AI? What strategies helped you get better results?
- Taking the idea for your app you want to create, go work with ChatGPT or Claude to craft a series of prompts that will help you towards building a product requirements document.

Listen to this section:

(a vibe coders meet up!)

When you're building software using AI, it's helpful to shift your mindset a bit. Instead of thinking of your AI assistant as a simple tool, consider it more like a developer you've just hired. Imagine you're responsible for managing a software development project in the real world, how would you approach it? You wouldn't simply say, "Hey, build me an app that tells me what to wear based on a photo of my wardrobe," and expect flawless results. Sure, you can "vibe-code" away with a prompt like that, but ultimately, if that's all the input you're going to do, you're going to run into serious issues. Instead, you should go through a thoughtful process, carefully outlining your requirements, gathering input from different stakeholders, and continuously refining your project as you go along.

In a real-world scenario, imagine you're tasked with developing a new e-commerce platform. Before you write even a single line of code, you'd first gather input from various stakeholders to ensure the software you're building aligns with their expectations and needs. Each stakeholder typically has distinct areas of interest and priorities.

- For instance, the business team might focus on core functionalities and revenue-generating features, such as seamless checkout experiences, robust inventory management, and integrated payment systems. They will want features that directly impact profitability, ease of use, and customer retention.
- Meanwhile, the marketing team would likely emphasize user acquisition strategies, user experience, branding consistency, analytics, and integration with social media channels. They'd be concerned with how easily they can track user behavior, run promotional campaigns, and convert visitors into paying customers.
- On the other hand, your finance team might focus on cost efficiency, budgeting, financial reporting capabilities, and scalable infrastructure. They will ask questions like, "How much will hosting cost?", "Are there more cost-effective solutions?", or "How do transaction fees affect our margins?" These considerations can directly impact which technologies you choose and how you design your platform to manage expenses effectively.
- Finally, your operations and infrastructure team would likely emphasize reliability, scalability, and security. Their priority would be ensuring that the app can handle peak traffic during sales or promotional events, that it's safe against cyber threats, and that customer data is managed securely and compliantly.

Each of these stakeholders' interests shapes the final form of your software significantly. By thoughtfully incorporating their insights, you ensure the resulting application is not just functional, but strategically aligned with your broader business goals, financially viable, and operationally sound. The more comprehensive your stakeholder engagement at this stage, the clearer your instructions become, and consequently, the higher quality of code and output you will receive from your AI assistant.

Here's the best part, you can simply ask the LLM to play the role of that stakeholder so they can bring their concerns forward for your specific app, and work on the code that will affect their requirements.

Now, bring that same disciplined approach into your interactions with an LLM. Rather than relying on brief or vague prompts, start your project by clearly defining your goals and breaking them down into manageable steps. Communicate these steps explicitly to the AI, just as you would brief a human developer. Each prompt should build logically upon the previous one, detailing exactly what you're looking for and providing context whenever possible.

This approach leads you toward creating a Product Requirements Document (PRD)—a comprehensive guide that outlines every aspect of your software project. A well-crafted PRD includes user stories, feature descriptions, technical constraints, budget considerations, and deployment plans. When you're using an LLM, your PRD might begin as a simple series of structured prompts and gradually expand into a robust set of instructions that guide the AI to generate higher-quality, more accurate code.

Consider an example: Mike, a small-business consultant, wanted an app to streamline appointment scheduling. Instead of simply prompting, "Make me an appointment app so that customers can make appointments with me by going to a web page," That's the bare minimum to get started, but you won't get very far with that.

Trying a different way: Mike approached his LLM with a detailed vision. He first described the core functionalities clearly: an easy way for customers to create appointments, calendar integration, automatic reminders via email and SMS, basic customer information, and payment integration. Then, he refined his prompts based on specific feedback from test users and colleagues, just like he would when working with a traditional development team. The end result? A basic app to begin with, that he could build upon in stages that would add one feature at a time getting to the overall desired goal.

You just can't build this all in one prompt and expect something amazing. You need to break it into phases, and add features one by one, making sure the app works well through the entire build process.

The key takeaway here is that the more precise, structured, and thoughtful your instructions, the better your results. Treat your LLM as if it were your employee—give it clear guidance, context, and iterative feedback—and you'll be amazed at what you can build together.

Questions to consider:

- If you hired a developer today, what information would you provide them before they began working on your app idea? Hint: start with that PRD!

- Draft a series of prompts to see how detailed they need to be? Where do you see gaps where more structure or clarity might improve your results? Stuck on what to write? Get your favourite LLM to help you and ask it for what you need!

Listen to this section:

Before you dive deeper into building your project, let's pause for a moment and consider something fundamental: what exactly is "good" software? You might initially think it's software that simply works without errors—and that's certainly important—but truly good software goes much deeper than functionality alone.

Think about apps or tools you use daily that you genuinely enjoy and rely upon. Maybe it's your favorite productivity app, streaming platform, or online shopping site. What sets these apart isn't just that they function; it's that they're intuitive, reliable, scalable, secure, and genuinely solve a real problem for users. They seamlessly fit into your life, making tasks easier and often even enjoyable.

When defining good software, start with the end-user in mind. A good application anticipates user needs and addresses pain points directly. For example, consider a budgeting app. A basic budgeting app might simply track income and expenses, and technically, it "works." But a great budgeting app goes beyond this—offering automated insights into spending habits, taking photos of receipts to input data rather than you typing anything, alerts for unusual transactions, and tailored recommendations for savings. It makes complex financial data understandable so the user knows what to do with that information.

To build that, you need to figure out several things:

- what language are you going to write it in? (hint: ask the AI for help on that)
- how does it need to be hosted? (hint: SPA (single page application), frontend/backend (with a database), storage, authentication (user accounts))
- how should users sign up?
- how much should it cost them? (in other words, how will you make money?)
- can you add a freemium version?
- and other questions for your PRD

The user experience is only part of the picture. Good software must also be scalable meaning it performs equally well whether it's used by ten people or ten thousand. Imagine launching your app successfully only to discover it crashes under heavier user traffic. Scalability considerations from the start ensure your app can gracefully handle growth without sacrificing performance.

Security is another critical aspect. Users trust software with personal data, from basic contact information to sensitive financial details. Good software earns and maintains this trust through robust security practices. Think back to our budgeting app example: if users are going to trust your app with their spending habits and transaction history, it needs encryption, secure authentication, secure data storage, and regular security audits to protect their data comprehensively. Furthermore, thanks to GDPR regulations in Europe, users need to be able to either download their data, have a way to delete their account, and ultimately have control over their content.

Lastly, good software is maintainable and cost-effective. It should be structured clearly so future enhancements, fixes, or new features can be efficiently added without rewriting everything from scratch. Good software also carefully considers ongoing operational costs like hosting, third-party API integrations, and maintenance efforts. Poorly constructed software, by contrast, can become financially unsustainable over time.

If you don't know what's involved, just ask the LLM!

We have upcoming course modules on scalability, security, and UX (user experience) on this platform that you will come to later to give you a great foundation. However, it always helps to simply ask for more information. Role play telling the AI to be a different person and get their assessment of things. The UI, the "flow" of an app, the security configuration, etc. The information you need is all there, you just need to be asking the right questions to get the AI to do what you need it to as it won't always remember or feel inclined to tell you.

Let's tie this back to your own project. As you work with your AI assistant, think beyond immediate functionality. Clarify these broader aspects of good software within your prompts and your overall plan. Provide guidance on user experience expectations, scalability requirements, security standards, and maintainability preferences. The clearer you define these qualities from the outset, the better your final product will be. All this helps drive you towards building that PRD which will be the blueprint for your application.

Questions to consider:

- Think of your favorite app or website. What makes it stand out compared to others? Why do you use it? What problem did it solve for you? Then ask, what problem are YOU trying to solve with your app?
- Are there examples of software you've abandoned because they lacked some of these essential qualities? What was it that turned you off that you need to be aware of in your own app idea?
- How clearly are you communicating expectations about scalability, security, and user experience to your AI assistant? Can you refine your prompts to better address these areas? If you don't know what these items are, just ask it!

Listen to this section:

When working with AI to build software, the quality of your results is directly tied to how clearly and thoroughly you communicate your ideas. Imagine again that you're managing a software developer in real life. You must provide specific guidance on exactly what needs improvement. The same approach applies here. The more detailed and context-rich your prompts, the better the AI's output will be. So let's dive deeper into how you can consistently write prompts that produce outstanding results.

Providing more context

Instead of "make the formatting look better", try, "On a mobile view, the page looks like it's been squashed a bit just below the hero section. We need to add more spacing below it. Here's a screenshot and I've drawn on it where we need to add more spacing."

First off, never underestimate the power of context. The AI assistant you're working with doesn't have access to your environment or your full vision unless you clearly describe it. If you're encountering an error, don't just tell the AI, "My app isn't working right. I get an error message." Instead, describe precisely what's happening: share the exact error messages, copy and paste code outputs, include screenshots with markup, or even capture video snippets demonstrating the issue in action.

Check out the installation page (link also in the Resources box on the Dashboard) on how to enable the javascript console to see realtime logging data.

Providing screenshots (which you then drag and drop into the chat)

Let's say you're developing a dashboard and the layout isn't rendering correctly. Instead of vaguely stating, "Dashboard looks weird," you might prompt the AI like this: "Here's a screenshot of my dashboard page. Notice the widgets overlap at certain screen sizes. Here's the CSS and React component code for this page. How can you adjust this to make it responsive?" For UI-related queries, provide annotated screenshots showing exactly what needs to be changed or fixed. When you give this kind of detailed input, your AI assistant can diagnose the issue and generate solutions far more accurately.

(see the screenshot attached to a chat)

(On a Mac, press Command + Shift + 4 to draw an area around where you want the screenshot and it will automatically save to the desktop. Windows Instructions are [here])(<https://www.awesomescreenshot.com/blog/screenshot/how-to-screenshot-on-windows>).

Additionally, if you find yourself stuck on a tough problem, don't hesitate to ask the AI to provide multiple solutions or approaches. A prompt like, "Give me three ways to fix this database connection error, and rate each solution on your confidence level for fixing this specific issue," can be tremendously helpful. You'll receive varied insights, allowing you to weigh your options carefully

before implementing a solution.

Another valuable tactic is to proactively explore solutions or technologies you might not be aware of. If you're unsure about which cloud database platform to choose, simply ask something like, "What cloud-based databases are suitable for securely storing user data? Can you highlight their main features, pros, cons, and typical costs?" Similarly, you might ask, "Are there reliable free SMTP mail servers available for handling my app's outgoing emails?" If you don't know what an SMTP server is, you could simplify your prompt even further. "We need to send out emails to people if they reach us via a contact form. How to we best implement a contact form and how will the emails go out?" Upon hearing you need an SMTP server, then you could continue to ask for any free options. These open-ended prompts often uncover valuable new tools, resources, or solutions you hadn't considered.

Ultimately, the clarity, thoroughness, and precision of your prompts shape your success when using AI to develop software. Treat your interactions thoughtfully, like conversations with a trusted, yet junior developer who requires clear, detailed guidance. The effort invested here directly translates to higher-quality, more robust, and reliable software. More importantly, you will spend less time on your app as you will get more done as part of your conversation.

For a lot more on this topic, read the Google Prompting Guide listed in the Resources section on your Dashboard.

Best Practices for Good Prompt Engineering:

1. **Provide Rich Context:** Always try to include detailed descriptions, screenshots, error logs, or code outputs. The more complete your context, the more accurate the AI's response will be. The less requests you use getting what you want.
2. **Break Down Complex Problems:** Split big issues into smaller, manageable prompts. Solve each part methodically to avoid confusion and errors. Have the AI create a long task list of what's needed and methodically work your way through it.
3. **Request Multiple Solutions:** When stuck, ask your AI assistant explicitly for several possible solutions, along with confidence ratings, to help you evaluate options clearly.
4. **Ask Open-ended Questions:** If you're uncertain about available technologies or solutions, use prompts like, "What are the available options for...?" This can uncover new tools or methods you weren't aware of.
5. **Pause and Reassess Regularly:** If solutions repeatedly fail, explicitly prompt the AI to pause, reassess, and clarify. Provide clear observations, additional logs, or debugging data to help it find the root cause.
6. **Use Specific Examples:** Reference other websites, apps, or features directly in your prompts. This gives clear benchmarks or visual goals for your assistant to target.
7. **Include Visuals with Annotations:** Screenshots with clear markup or annotations dramatically enhance clarity, especially for UI/UX tasks. This can save you significant hours wasted (and usage requests)

8. Confirm AI Understanding: Occasionally ask the AI to summarize your instructions back to you. This ensures alignment and catches misunderstandings early.
9. Explicitly Mention Constraints: Clarify limitations upfront, such as budget constraints, preferred technologies, or deadlines, to get more practical, relevant solutions.
10. Test and Validate Thoroughly: Always verify changes proposed by the AI. Carefully test major modifications to ensure no unintended side effects or regressions occur.

Questions to consider:

- Think about a recent interaction you had with AI. Were your prompts as detailed and clear as they could have been? What could you have improved?
- How might you structure your prompts differently when dealing with complex debugging or exploring new technologies?
- Can you identify specific strategies or tools (such as screenshots, logs, or examples from other sites or apps) that you might use regularly to improve your prompt effectiveness?

Listen to this section:

Continuing from our previous section on good prompts, let's talk about an essential strategy for getting the absolute best out of your AI assistant: actively challenging the solutions and ideas it provides.

Just like working with a human developer, your AI assistant's suggestions might initially seem great but without deeper scrutiny and proactive questioning, they might miss important considerations. Encouraging your AI to critically evaluate both your ideas and its own proposals can uncover significant potential issues and lead to stronger, more robust solutions.

One common pitfall users face is getting trapped in a cycle of trying changes, finding they don't work, and either rolling them back, or leaving the code there but without gaining ground. This is common when you're trying to implement some major changes. Or diagnosing some security errors in the logs. First off, just take a deep breath.

Now, prompt the AI clearly: "I've tried these changes, and they're not working. Let's stop and reassess. Can you help me figure out why these solutions haven't worked? Please suggest additional options and give me your confidence level in them." Encouraging the AI to stop and take stock helps break frustrating cycles and usually reveals deeper problems or simpler solutions.

When you do fix your issue, you might introduce a new one, so in light of that it's crucial to remember that your AI assistant, while incredibly powerful, isn't flawless. AI can forget previous context, make assumptions, or occasionally introduce mistakes. It may sometimes remove essential code when editing or accidentally introduce bugs. Therefore, don't just blindly accept every suggestion. Thoroughly test each solution, especially major changes, to ensure nothing else has inadvertently broken. If something does go wrong, clearly communicate this back to your AI assistant, offering precise feedback so it learns to improve its next suggestions.

Here's another example: Imagine you're building a user-authentication flow. The AI suggests altering your authentication logic. Don't immediately assume it's perfect—carefully test all scenarios: user registration, login, session management, error handling, and security checks. If anything fails, document the issue and explain precisely back to the AI: "Your last edit caused new registrations to fail with this error message. Here's the detailed log output and related code. Can we carefully revisit and fix this?" This method ensures accuracy and reliability in your final product.

In this video, here's an example of Cursor changing something I didn't ask for!

Cursor can be naughty!

An example from one of my previous apps.

In a different app I built, I have a user dashboard. In this app you submit a link to a social media video and the app processes it for information. This processing with AI takes about 30-40 seconds. Your AI assistant suggests implementing a "polling" system, where the dashboard repeatedly checks (every few seconds) to show if the data is ready and changes an icon from processing > completed. At first glance, this seems perfectly reasonable, especially during testing or development.

But here's where challenging your AI assistant becomes crucial. What happens when your app moves from testing to production, with potentially thousands - or even tens of thousands - of users simultaneously using this feature? While polling might work smoothly during your testing phase, it can create enormous performance bottlenecks and scalability issues in a real-world environment. Frequent polling requests from thousands of concurrent users could overload your server, degrade performance, and negatively impact the user experience.

In this scenario, it's essential to prompt the AI to recognize your application's scale requirements explicitly. You might say something like, "Polling is great in a small test environment, but what about when we scale this to 10,000 simultaneous users? Could this approach cause performance issues, and what alternatives could better support this level of usage?" This kind of proactive questioning pushes your AI assistant to consider scalability concerns, prompting it to offer alternative solutions—such as WebSockets or server-sent events (SSE), both of which can provide real-time updates more efficiently at scale. I wasn't aware about WebSockets at the time, but it turned out to be a great suggestion to handle the volume.

Think about your mobile users

Another critical area to challenge is User Interface (UI) and User Experience (UX) design. Suppose the AI assistant suggests changes or additions to the desktop layout of your app. It's important not to automatically assume these changes translate seamlessly to mobile devices. The AI might not automatically think about the mobile view unless explicitly instructed. Therefore, proactively ask, "How does this design change impact the mobile view? Can you ensure that this solution maintains usability and visual consistency across desktop and mobile devices?" This ensures your AI takes a more holistic and responsive approach to UI/UX design, helping you maintain a high-quality experience across all devices.

Obviously, you can test this yourself but you don't have every phone to use, just your own.

Encouraging your AI assistant to critique its own proposals is equally valuable. You could instruct it: "Here's my idea: can you identify any potential problems or unintended consequences I might not have considered?" or "You've proposed this approach; what are some potential weaknesses or scalability concerns with your solution?" By explicitly requesting this type of analysis, you not only uncover hidden issues early but also reinforce good development practices that emphasize thoroughness, anticipation of potential pitfalls, and strategic foresight.

Ultimately, challenging your LLM consistently pushes it beyond surface-level answers, guiding it toward more thoughtful, robust solutions. This practice of mutual critique and questioning not only builds better software but also sharpens your own strategic thinking and development skills.

Best Practices for Effectively Challenging Your LLM:

1. **Explicitly State Scale and Performance Goals:** Clearly mention your production goals, such as user capacity or performance requirements, to prevent short-sighted solutions.
2. **Ask for Critical Feedback:** Regularly prompt your LLM to evaluate and critique your ideas and its own solutions to catch potential issues early.
3. **Encourage Alternative Solutions:** When the AI suggests a solution, actively request other methods, particularly for scalability and performance considerations.
4. **Maintain a Mobile-first Perspective:** Ensure your LLM is reminded to consider responsive design, explicitly prompting it to evaluate desktop solutions against mobile compatibility.
5. **Clarify Long-term vs. Short-term Solutions:** Clearly distinguish between testing solutions and production-ready solutions, prompting your AI to recognize and address long-term requirements.

Questions to consider:

- Have you encountered scenarios where AI-generated solutions worked well in testing but failed in real-world conditions? How could clearer prompts have prevented this?
- Do you regularly challenge the AI to justify its recommendations, particularly around scalability, security, or compatibility issues?
- How can you consistently prompt the AI to think holistically about both desktop and mobile user experiences when developing UI solutions?

07. Start with something small before you build your first Software as a Service (SaaS) product. 3 min

Listen to this section:

Let's shift gears a bit and talk about Software as a Service, often called SaaS. This is software that users typically access online through a subscription model rather than buying it outright with a one-time payment. Building a SaaS product is an exciting step and can be highly profitable, but it introduces additional layers of complexity that you'll need to thoughtfully navigate. As you start to plan your SaaS product, it's crucial to consider aspects beyond just coding and technical functionality. What is great is that there's a new layer of SaaS that has developed that's one step below a full SaaS product, that's Micro SaaS. Have a look at the table below to understand the differences.

It's quite likely that you will have ideas that fall into the micro saas category for many reasons listed in the table. If you're a one-person business, this will suit you very well and Cursor is one of the many tools you'll need for your journey. But there are lots of other things you need to keep in mind as you move to the idea of running a business from software alone.

So first, start off with something relatively easy, like an SPA. That's a Single Page Application. Almost everything runs on one page - the landing page. It's got a hero section, pricing, FAQ, contact area, navigation, and a footer. Get used to that as your first project! Then go ahead and deploy to a place like Vercel or Railway. Learn the ropes on something easy rather than something difficult! Then you will have so much more confidence to attack a more complicated project the next time around.

Examples of SPAs would be a personal website for an artist, a small business (plumber or home renovation business), a recruitment agency, etc.

Here's an example for one I built from start to finish in 2 hours (including the deployment to Vercel). That 2 hours is to a finished product. When I was first learning, it probably took me a week or two! But after practice, and learning, and more powerful models, the process is getting so much easier. I charge anywhere from £800-£1300 to create a site like that plus monthly hosting fees. They're simple so you don't have as many security concerns. They don't store any users's data. And they're great for you to build a fake portfolio to show clients if you don't have any! It has the following features:

- mobile and desktop compatibility
- google analytics compatibility
- github integration integrated into Vercel
- responsive design
- serverless architecture
- contact form with custom SMTP server
- built on React with Typescript

- performance optimisation with code splitting and fast load times

These can be built in a day and what's nice about them is they are easy to deploy too! Get something up and then start on something more complex. Trust me you will learn so much from doing the whole process from start to finish with an SPA that it will give you a small taste of what's to come. Then you can up-sell include add-ons with some basic workflow automation. How about a make.com automation that intercepts the contact emails and responds with more questions for patient discovery that gets put into a spreadsheet? That spreadsheet is then summarised daily and sent as a digest?

If you treat it like a proper project, your next "project" being the first phase of your micro SaaS app will seem a bit easier to approach.

The SaaS product is going to be something you sell to others, so it's going to have some key requirements you'll need to think about before you get started. Before the nitty gritty boring stuff though, here's a high-level overview of your approach.

1. Ask yourself what problem you're trying to solve?
2. Who would be your customers?
3. How would the product work?
4. Is there any competition/demand?
5. What do you need to get to an early prototype?

Those 5 items are the bare minimum to get yourself off the ground. This is where you need to spend the most time hashing this information out.

As you get closer to stage 5, then you can focus on some of the additional items below as part of your overall understanding and preparation. These aren't things that have to be completely ironed out on day one but will need to be considered when you launch.

First, think about the legal side. Unlike simple apps you might create for personal use, SaaS products often handle sensitive user data: names, emails, financial details, and perhaps even more private information. You'll need to consider regulations like GDPR in the European Union, or CCPA in California, which strictly govern data privacy.

Moreover, you need to think about user obligations and the agreements they must accept. Clearly outlining terms of service, acceptable use policies, refund policies, and cancellation terms will protect your business and set clear expectations with your users.

Financial considerations become much more significant when you're running a SaaS business. How will you accept payments? How do you manage subscriptions, handle renewals, or offer upgrades and downgrades? If your business attracts international customers, how will you handle multi-currency payments?

Speaking of finances, taxation is another key factor. Running a SaaS business often means dealing with various tax obligations. Depending on where your customers live, you might need to charge different sales taxes, VAT, or digital services taxes.

Another critical consideration is your licensing model. Traditional software licensing often involves a perpetual license, customers pay once and use the software indefinitely (though perhaps paying separately for upgrades). SaaS products, on the other hand, typically follow a subscription licensing model, where users pay monthly or annually for continuous access.

Platforms like Stripe for payments and Xero for managing your finances will help you massively without the need to hire other people. Of course, the AI assistant will help you implement the Stripe requirements directly into your site (we have a module on this topic later). Stripe help also has its own very good AI assistant too!

Questions to Consider:

- Have you defined a problem you're trying to solve?
- How would that problem be solved with your software?
- What do you need to get it off the ground?

Listen to this section:

Here's one of the biggest roadblocks for aspiring software builders: perfectionism. It's incredibly tempting to keep polishing, tweaking, and fine-tuning your app until every pixel feels just right, especially when you're building something you care deeply about. As you build things, you realise it would be great to add this feature, add that feature. Add something else shiny you saw on another website. The list can be endless frankly!

But here's the truth: if you're the only person using it, you're not building a product, you're building a pet project.

You don't need to wait until your software is "finished" (spoiler: it never really is). What you need is to ship something. Something real. Something users can interact with. That's how you find out whether your idea actually has legs by testing product/market fit in the real world. Until that happens, you're operating on guesswork. This very website was shipped well before all the modules were completed, and with a few missing features from the site. It will also be ok though as it doesn't have to be perfect.

Instead of building the entire vision upfront, focus on developing a Minimum Viable Product (MVP). This is a lightweight version of your app that includes only the core functionality necessary for early users to experience the main value you're offering. It's your proof of concept, the spark that lights the fire.

Let's say you're building an AI-powered resume builder. Rather than adding every feature under the sun: job-specific templates, cover letters, portfolio links, analytics, etc. your MVP might focus on generating a single high-quality resume from user input. That alone is enough to get feedback, test your core premise, and prove demand. And once you see people getting value from that MVP, you'll feel way more confident in continuing to build.

If you're seeking investors, getting to this point is also good to show them a working prototype. It gives them a chance to get a look and feel for the solution and understand you're really building this and it's not just smoke in the air.

At this stage, consider launching a beta program. Offer users early access in exchange for feedback or a reward, like discounted or free access once the platform officially launches. Reach out to potential users where they already hang out: industry forums, Discord groups, Reddit threads, Twitter (X), niche Facebook communities, even in-person meetups. Wherever they are, go there.

You can also host live demos or book short intro calls with users. Walk them through the product and showcase what you've built so far. Not only does this help you gather valuable insights, but it shows your users that you're actively listening and that you're building something with them, not just for them.

Don't forget to promote your beta program on social media as well. Invite people to sign up and help shape the future of your product. You might be surprised by the interest you generate and you'll definitely learn more than you would by just building in a vacuum. Of course, don't forget to ask the AI to help you build out your marketing and social media plan with daily posts and activities for the various sites out there.

From a technical standpoint, you'll want to build in some infrastructure to support this beta phase. One great approach (which we'll cover in more depth later) is to use an automation flow: let users sign up via a form, automatically add their email to a Google Sheet, and then push that info into a `beta_whitelist` table in your backend. This allows you to restrict access only to beta testers while keeping the general public out for now.

Here's my workflow in another app I have that does this exact process using make.com that's super simple to setup!

Within your app, give testers a clear way to provide feedback. A contact form where they can attach screenshots and describe issues or ideas is incredibly useful. You should also track enhancement requests and bugs. This is the goldmine that tells you what users actually want.

To make the most of this phase, consider building a simple internal dashboard to monitor key usage metrics. See which features get the most love. Identify potential bottlenecks. Track usage of paid APIs or third-party services like LLMs or scraping tools. Sometimes these costs can sneak up on you if usage spikes during beta.

Ultimately, your beta phase is a safe space to learn, adapt, and grow your product with real users, real data, and real stakes. It helps you answer the most important question of all: "Is this thing actually useful to the people I built it for?"

Best Practices for Shipping and Running a Beta Stage:

1. Don't chase perfection—ship your MVP early. You'll learn far more from real users than from continued solo polishing.
2. Define and limit your scope. Focus on building just enough for users to see value and validate your core idea.
3. Set up a structured beta program. Offer free access or perks in exchange for feedback and early usage.
4. Promote where your users already are. Find communities or platforms where your target audience spends time.
5. Host calls or live demos. Walk people through your product and gather real-time feedback.
6. Use an automation flow for onboarding. Streamline beta access using tools like Google Sheets and backend whitelisting.

7. Provide easy ways for users to give feedback. Include screenshot upload forms, bug trackers, and enhancement request features.
8. Monitor feature usage and resource load. Build a simple dashboard to track where your product is being used most—and where it might cost you.
9. Plan for scalability as your beta grows. Watch how your system handles user load and make adjustments gradually.
10. Keep iterating based on real feedback. Every insight from a beta user is an opportunity to make your product better.

Questions to Consider:

- What's the minimum version of your product that still delivers value to users?
- How will you find and recruit your first beta testers?
- Are you collecting feedback in a structured way that helps you make informed decisions for your next build?

Listen to this section:

Let's be real: this journey you're on, building software with AI, especially if you're new to coding is going to test your patience. There will be moments of excitement and flow, yes, but there will also be days where it feels like you're running in circles. One fix leads to another issue. A feature that worked yesterday mysteriously breaks today. And sometimes, despite your best efforts, a bug will linger for hours - or days - before you finally crack it.

This is all completely normal.

Debugging is part of the process, and it's one of the most humbling and instructive aspects of software development. Think of it like solving a puzzle, except sometimes the pieces change shape as you're fitting them together. It's not uncommon to take one step forward only to stumble two steps back. But each stumble is an opportunity to learn something that will make you sharper the next time around.

Let's say you're trying to integrate a new authentication system. You finally get the login flow working: users can sign in, sessions are created, and everything feels good. Then you notice your profile pages are broken. Turns out, the logic that checked if users were logged in was depending on the old session model. Now you're off fixing that, which leads to yet another issue with protected routes. This is how it goes. Fix one thing, and another area may break because of a dependency you didn't realize was there.

It's frustrating. But it's also incredibly rewarding when you finally get it all working. That "aha!" moment after hours of debugging feels amazing like lifting a fog and suddenly seeing clearly.

If you're feeling stuck, consider reaching out to someone you know who works in tech. Seriously, buy them a coffee. Pick their brain. Share what you're building. Developers who've been through the wars of production software can offer you perspectives and shortcuts that no tutorial or forum can replace. Ask them about the worst bugs they've had to fix or how they manage complex systems. You'll likely walk away with a better understanding of your own project and maybe even a new debugging strategy to try.

And if you're the kind of person who thrives on structure, good news: you can bring that mindset into your workflow. Treat your software project like a checklist of small tasks. Some need to be done sequentially, while others can happen in parallel. Break big features down into smaller components like pages, buttons, API calls, validation rules. Ticking off those small wins one at a time not only builds momentum, but also helps you stay focused and avoid feeling overwhelmed.

Real-world example? A friend of mine was building a task management app with AI-generated reminders. It started off smooth with simple task creation, UI design, backend routes. Then came the intelligent reminder feature, where the app suggests when to do certain tasks based on user history.

Suddenly, timezone logic broke date formatting, which caused the reminders to fire at the wrong times. Fixing that broke the calendar view. Then the reminder email flow started firing twice. Every fix introduced a ripple effect. It took nearly a week to get it all behaving properly. But they learned more that week than they had in the previous month.

You don't need to be perfect, and you don't need to solve everything immediately. You just need to stay patient, curious, and willing to keep showing up.

Best Practices for Staying Sane and Steady During Debugging:

1. Expect setbacks—it's part of the job. Don't panic when something breaks. Assume it will, and be ready to investigate.
2. Keep your debugging process methodical. Work one problem at a time. Reproduce the issue. Log everything. Check assumptions.
3. Don't debug in silence. Talk it through with a dev friend or even just out loud—it helps more than you think.
4. Use AI as a debugging buddy. Share logs, describe behavior, and ask the AI for likely causes or strategies to isolate the issue.
5. Talk to experienced developers. A coffee chat with someone who's been there can give you shortcuts you won't find online.
6. Lean into structure and task planning. Break your project into tiny, doable tasks and check them off as you go.
7. Be careful with "fixes" that create side effects. Always test after changes. What looks fixed in one place may be broken in another.
8. Celebrate small wins. Every bug you squash makes you a better builder.

Questions to Consider:

- How do you handle frustration when something isn't working? Do you take a step back or try to power through?
- Are you building out your project in small, manageable steps or trying to solve everything at once?
- Do you have someone in your network who could give you a fresh perspective on a tricky issue? When's the last time you reached out?

- AI significantly lowers the barrier to entry for software development. You no longer need to be a seasoned developer to bring your ideas to life. With tools like Cursor and LLMs, you can build meaningful software even if you're just starting out.
- Treat your LLM like a junior developer. You're the project lead so communicate like one. Give clear direction, provide context, and refine your prompts as you go. The AI is powerful, but it works best when guided with precision.
- Building software still requires discipline and structure. AI can write a lot of code quickly, but without a process, that speed can lead to bloated, unstable projects. Approach your AI-assisted builds the same way you would with a real team so break tasks down, create plans, and follow best practices.
- Good software is more than just code that runs. It's secure, scalable, intuitive, maintainable, and provides real value to users. Think holistically like UX, performance, and long-term sustainability all matter.
- Prompt quality = output quality. Detailed, context-rich prompts lead to far better results. Share screenshots, code snippets, logs, use cases, whatever gives the AI a clearer picture of what you need.
- Challenge your AI and your own ideas. Don't just accept the first answer. Ask the AI to consider edge cases, point out flaws, and offer scalable alternatives. The more you question, the better your solutions will be.
- Don't wait for perfection, just ship. Launching early with a Minimum Viable Product (MVP) allows you to test your idea, collect real feedback, and build momentum. Perfection can come later but validation must come first.
- Use a beta program to validate and improve. Create a feedback loop with early users. Let them test your product, report bugs, and suggest enhancements. Use automation flows to manage access and capture data efficiently.
- Debugging is part of the journey so be patient. Expect some days to feel like two steps back. Fixes often reveal new issues. This is how real software is built. Stay curious, methodical, and willing to reach out to others when you're stuck.
- Structure helps you move forward. Treat your project like a task list. Break it down into bite-sized goals. Use that structure to manage progress, track bugs, and avoid overwhelm.
- Real-world input is invaluable. Whether it's feedback from beta users, coffee chats with devs, or insight from your target audience so talk to people. Learn from them. Build with them.
- Cursor is a uniquely powerful development tool for non-coders. It combines AI integration, real-time coding, and a user-friendly interface designed to make building software more accessible than ever before.
- The magic is in the feedback loop. Build, test, learn, refine. With AI by your side and a strategy in place, you're well-equipped to go from idea to product and beyond.